

simple, small collections of fields that should be treated in a lightweight manner and do not require all of the flexibility that classes provide. In fact, all primitive datatypes of C# are actually implemented as structs underneath the covers. For example, the declaration of an integer `int x = 42;` really declares `x` as value type `System.Int32`.

3.2 Boxing and unboxing

`int` is a value type. Thus its representation in memory is exactly the bits for the integer and nothing more. However, we can write code like this:

```
public class IntAsObjectTest
{
    public void Test()
    {
        // 3 calls to ToString for ints
        int x = 42;
        Console.WriteLine("{0}", x);
        x.ToString();
        42.ToString();
    }
}
```

That is, we can treat ints as objects even though they're value types. A more striking example is pre-generic containers such as `ArrayList`. In order to be "generic", non-generic `ArrayList`s can only hold references to objects. However, we can do add ints to `ArrayList`s as if they were objects.

```
public class ArrayListIntTest
{
    public void Test()
    {
        ArrayList l = new ArrayList();
        l.Add(42);
    }
}
```

Both of these things are possible are due to boxing. There are many times when you want treat a value type as an object. Boxing wraps a copy of the value type in a wrapper reference class that can call methods or be treated as a reference type. As seen above, this happens automatically.